

M2M Vector Search

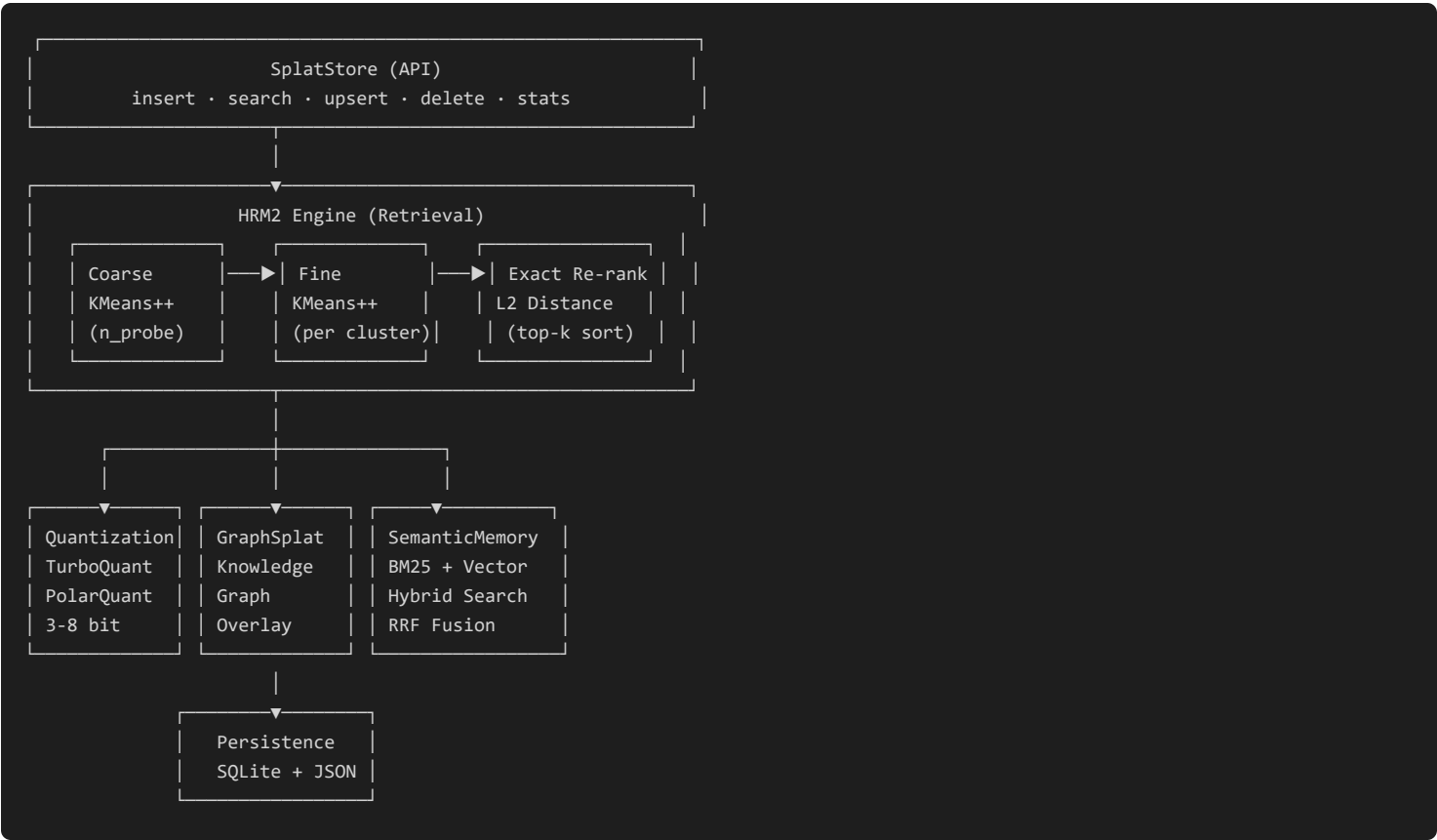
version2.1.0

licenseAGPL-3.0

rust2021

Native Rust Gaussian Splatting vector search engine — representing embeddings as probability distributions (Gaussian splats) instead of point vectors, enabling richer similarity semantics through two-level hierarchical retrieval (HRM2), compressed quantization, and knowledge graph overlays.

Architecture



Features

- **Gaussian Splat Embeddings** — each vector is a probability distribution (mean + alpha + kappa), not a point. Captures uncertainty and density.
- **HRM2 Two-Level Retrieval** — coarse KMeans++ partitions the space, fine KMeans++ refines within each partition. Configurable probe count controls recall/speed tradeoff.
- **TurboQuant / PolarQuant Compression** — 3-8 bit data-oblivious quantization. No training or codebooks required. 4-10x memory reduction for approximate search (theoretical; not yet benchmarked in Rust).
- **GraphSplat Knowledge Graph** — overlay graph structure on splats with typed nodes (Document/Entity/Concept), directed weighted edges, and graph-augmented retrieval (BFS traversal, relationship queries).
- **Semantic Memory** — hybrid BM25 + vector search with Reciprocal Rank Fusion (RRF). Auto-categorization of memories. Designed for AI agent memory systems.
- **KMeans++ Clustering** — deterministic initialization with ChaCha8 RNG, Lloyd's algorithm, Rayon-parallelized distance computation.
- **CPU-Optimized Engine** — parallel L2 distance with rayon, partial sort for top-k (O(n) average), ndarray einsum for batch operations.
- **CUDA GPU Acceleration** — custom PTX kernels for L2/cosine distance + top-k selection in one pass. Persistent VRAM dataset (no re-upload between queries). 7.8x speedup over CPU on RTX 3090. Feature-gated (--features cuda), CPU fallback always available.
- **SQLite Persistence** — full database storage for semantic memory with metadata, categories, and timestamps.
- **Configurable Profiles** — M2MConfig::simple() for edge devices, M2MConfig::advanced() for agentic workloads. Auto device detection (CUDA > Vulkan > CPU).

Energy-Based Model (EBM)

The EBM layer treats the vector space as an **energy landscape** where each Gaussian Splat acts as an attractor basin. This enables three capabilities that point-vector systems cannot provide:

Energy Landscape (`energy.rs`)

Each point in embedding space has an energy value:

$$E(x) = -\log(\sum_i \alpha_i * \exp(-\kappa_i * ||x - \mu_i||^2))$$

- **Low energy** = high confidence (near splat centers, dense regions)
- **High energy** = uncertainty (sparse regions, knowledge gaps)

Exploration (`ebm/exploration.rs`)

- **Boltzmann sampling**: Sample points from high-energy regions to discover where the model is uncertain
- **Knowledge gap detection**: Identify clusters of unexplored energy space
- **Exploration suggestions**: Generate queries and regions worth investigating
- **Greedy clustering**: Group high-energy points into actionable regions

Use case: An AI agent can ask "what don't I know?" and the EBM returns regions of embedding space with no coverage — guiding active learning.

Self-Organized Criticality (`ebm/soc.rs`)

- **Criticality monitoring**: Track whether the system is subcritical, critical, or supercritical
- **Avalanche detection**: When energy builds up in regions, trigger cascading reorganization
- **Natural reorganization**: Splats redistribute themselves through relaxation dynamics

Use case: Memory systems that self-reorganize when overloaded — preventing stagnation and maintaining retrieval quality over time.

Quick Start

Prerequisites

- Rust 1.56+ (Edition 2021)
- C compiler (for SQLite bundling via `rusqlite`)
- **For GPU acceleration**: CUDA Toolkit 12.x + MSVC cl.exe (nvcc compiles PTX kernels at build time)

Build & Test

```
# Clone
git clone <repo-url> m2m-rust && cd m2m-rust

# CPU-only build (no GPU deps)
cargo build --release
cargo test --lib                                # 211 tests

# CUDA build (requires CUDA Toolkit + MSVC)
$env:CUDA_PATH = "C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4"
cargo build --release --features cuda
cargo test --lib --features cuda                # 214 tests (3 GPU tests added)
```

Usage

```
use m2m_vector_search::{M2MConfig, SplatStore};

fn main() {
    // Use defaults (640D, 100K max splats, CPU)
    let config = M2MConfig::default();

    // Or edge profile: no 3-tier memory, no Langevin dynamics
    // let config = M2MConfig::simple(None);

    let mut store = SplatStore::new(config);
```

```
// Insert vectors
let embedding = vec![0.1f32; 640];
let id = store.insert(&embedding);

// Search for nearest neighbors
let results = store.search(&embedding, 10);
for r in &results {
    println!("id={} dist={:.4}", r.index, r.distance);
}
}
```

Module Reference

Module	File	Description
config	src/config.rs	M2MConfig with presets (simple/advanced/default), device detection
splats	src/splats.rs	SplatStore — main public API for insert/search/upsert/delete
hrm2_engine	src/hrm2_engine.rs	HRM2Engine — two-level hierarchical retrieval (coarse→fine→exact)
engine	src/engine.rs	M2MEngine — CPU-optimized L2 distance computation with Rayon
quantization	src/quantization.rs	TurboQuant/PolarQuant compression (3-8 bit)
quant	src/quant/	Integrated quantization algorithms: TurboQuant, PolarQuant, QJL, rotation (MIT, originally from turbo-quant)
clustering	src/clustering.rs	KMeans with KMeans++ init, parallel Lloyd's algorithm
graph_splat	src/graph_splat.rs	GaussianGraphStore — knowledge graph overlay on splats
semantic_memory	src/semantic_memory.rs	SemanticMemoryDB — hybrid BM25+vector search with RRF fusion
splat_types	src/splat_types.rs	Core types: GaussianSplat, SplatEmbedding, SplatCluster
geometry	src/geometry.rs	Sphere normalization and geodesic distance
encoding	src/encoding.rs	Full embedding builder (mu + alpha + kappa → flat vector)
bm25_index	src/bm25_index.rs	BM25 text search index
storage	src/storage.rs	SQLite persistence layer
energy	src/energy.rs	Energy landscape: $E(x) = -\log(\sum \alpha_i * \exp(-\kappa_i * x - \mu_i ^2))$. Lower energy = higher confidence near splat centers. Used by EBM for exploration and self-organization
gpu	src/gpu/	CUDA GPU acceleration — custom PTX kernels (distance.cu) for L2/cosine distance + top-k. GpuIndex for persistent VRAM datasets. CPU fallback when CUDA unavailable
hnsw_index	src/hnsw_index.rs	HNSW approximate nearest neighbor index
lsh_index	src/lsh_index.rs	Locality-Sensitive Hashing index

Performance

Measured Benchmarks — Rust Native (RTX 3090)

All benchmarks run on: **AMD Ryzen 5 3400G, 32GB DDR4, NVIDIA RTX 3090 (24GB)**. Compiled with `--release`, CUDA 12.4, custom PTX kernels.

GPU Top-K Search (Custom CUDA Kernels)

Combined distance + top-k selection kernel. Dataset persists in GPU VRAM between queries — only top-k results downloaded (not the full distance matrix).

Dataset	Dim	Queries	k	CPU QPS	GPU Persistent QPS	Speedup
10K	640	100	10	269	1,667	6.2x
100K	640	100	10	214	1,667	7.8x

Breakdown (100K vectors, 640D, 100 queries, k=10):

Phase	Time
CPU total	467ms
GPU upload (244MB)	62ms
GPU query (persistent)	60ms
GPU total (upload + query)	122ms

Throughput: 3.9 GB/s upload bandwidth, 0.60ms per query with dataset in VRAM.

CUDA Kernel Architecture



Kernel optimizations: - float4 vectorized loads (640D → 160 × float4 per vector) - Shared memory query cache (avoids global mem reads per thread) - Thread-local sorted top-k with shared memory merge - __launch_bounds__(256) for sm_86 occupancy - PTX compiled with --use_fast_math -O3 - Only top-k results downloaded (not full N×Q distance matrix)

CPU Performance (Baseline)

Operation	N	Dim	Time	QPS
Batch L2 search (100 queries)	10K	640	372ms	269
Batch L2 search (100 queries)	100K	640	467ms	214

GPU vs CPU — Feature Comparison

Feature	CPU (default)	GPU (feature-complete)
Search backend	ndarray + rayon	Custom PTX kernels
Dataset storage	RAM	VRAM (persistent)
Top-k selection	CPU partial sort	GPU thread-local merge
Cosine distance	ndarray	PTX kernel
Batch operations	rayon parallel	CUDA streams
Dependencies	base (10)	+ cudarc, libloading
Compile time	~5s	+PTX compilation (~2s)
Tests	211	214 (+3 GPU tests)

Build Modes

```
# CPU-only (no GPU deps)
cargo build --release

# With CUDA (requires nvcc + MSVC cl.exe in PATH)
CUDA_PATH="C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4" \
cargo build --release --features cuda

# CLI benchmark
cargo run --release --features cuda -- bench-gpu -n 100000 -d 640 -q 100 -k 10
```

Python Reference (Not Measured)

No Rust-specific benchmarks are available for HRM2 retrieval. The Python prototype showed 32-96x speedups with HRM2 vs linear scan, but **these numbers do not apply to Rust** and are not included here.

Configuration

```
use m2m_vector_search::config::{M2MConfig, QuantAlgorithm, SearchBackend, FusionMethod};

// Six presets:
let general      = M2MConfig::default();                // balanced
let edge         = M2MConfig::simple(None);              // minimal
let agentic      = M2MConfig::advanced(Some("cuda"));   // full power
let research     = M2MConfig::training(None);           // embedding research
let cluster      = M2MConfig::distributed(None);        // multi-node
let gpu_accel    = M2MConfig::gpu(None);                // CUDA-accelerated
```

M2MConfig::default() — General Purpose

Balanced config for servers and workstations. TurboQuant compression, GraphSplat knowledge overlay, and hybrid BM25+vector semantic search are all **active by default**.

Subsystem	Status	Key Parameters
Search	HRM2 (coarse→fine→exact)	knn_k=64 , hrm2_n_coarse=16 , hrm2_n_fine=64 , hrm2_n_probe=4
Quantization	✔ TurboQuant 8-bit	quant_bits=8 , quant_projections=160 , quant_search_fraction=0.1
Graph Overlay	✔ GraphSplat	graph_max_neighbors=10 , graph_traverse_depth=3 , graph_boost_weight=0.05
Semantic Memory	✔ Hybrid BM25+Vector	semantic_fusion=RRF , bm25_k1=1.5 , bm25_b=0.75 , decay_halflife=24h
Indexing	MapReduce	mapreduce_n_chunks=8
Quality	Reflection ON	quality_recall_target=0.95
Energy/EBM	✔ Splat + Geometric	energy_splat_weight=1.0 , energy_geom_weight=0.1 , soc_threshold=0.8
Langevin	✔ 200 steps	langevin_dt=0.001 , langevin_gamma=0.1
Memory	3-tier (hot/warm/cold)	max_splats=100,000 , context_windows=12/64/512
HNSW / LSH	✗ Off	Available but not needed at this scale
Auto-Scaling	✗ Off	Single-node deployment

Use when: general search applications, research, prototyping, production single-node.

M2MConfig::simple(device) — Edge / Embedded

Stripped-down for resource-constrained environments. Pure HRM2 search, no compression overhead, no graph, RAM-only.

Subsystem	Status	Change from Default
Search	HRM2 only	knn_k=32 , hrm2_n_coarse=8 , hrm2_n_fine=32 , hrm2_n_probe=2

Subsystem	Status	Change from Default
Quantization	❌ Disabled	No compression overhead
Graph Overlay	❌ Disabled	No knowledge graph
Semantic Memory	❌ Disabled	No BM25, no fusion
MapReduce	❌ Disabled	Single-chunk indexing
Quality	❌ Disabled	No recall tracking
Langevin	❌ 0 steps	No dynamics refinement
Memory	RAM-only	max_splats=10,000 , n_splats_init=1,000
Energy/EBM	❌ Off	No energy computation
Context	Minimal	8/32/128 vs 12/64/512
MoE	Reduced	2 experts / 1 active vs 4/2

Use when: Raspberry Pi, IoT, embedded systems, <4GB RAM, no disk I/O budget. Peak throughput with minimal footprint.

M2MConfig::advanced(device) — Agentic AI Workloads

Everything enabled and tuned for autonomous agents with self-organizing memory, aggressive compression, and distributed scale.

Subsystem	Status	Change from Default
Search	HRM2 + HNSW secondary	knn_k=128 , hrm2_n_coarse=32 , hrm2_n_fine=128 , hrm2_n_probe=8
Quantization	✅ TurboQuant 4-bit	quant_bits=4 , quant_projections=80 , search_fraction=0.2
Graph Overlay	✅ GraphSplat deep	max_neighbors=20 , traverse_depth=5 , boost_weight=0.1
Semantic Memory	✅ RRF fusion	vector_weight=0.6 , bm25_weight=0.4 , decay_halflife=1h
HNSW	✅ Secondary index	M=32 , ef_construction=400 , ef_search=100
MapReduce	✅ 16 chunks	Parallel bulk indexing
Auto-Scaling	✅ 2–20 nodes	cooldown=30s
Quality	✅ 99% recall target	Near-exact retrieval guarantee
Energy/EBM	✅ Full	Knowledge gap detection, self-organized criticality
Langevin	✅ 200 steps	Continuous splat refinement
Memory	3-tier, 1M splats	n_splats_init=50,000 , context=24/128/1024

Use when: persistent AI agents, autonomous memory systems, long-running chatbots, distributed deployments. Requires ≥8GB RAM. GPU recommended for 1M+ splats.

M2MConfig::training(device) — Embedding Research & Model Development

Starts from default, then reconfigures for model training. Disables search-heavy features (quantization, graph, semantic memory) to focus compute on training loops. Adds data lake management, noise robustness, Matryoshka multi-scale learning, and knowledge distillation.

What you get from default: HRM2 search, 3-tier memory, Langevin dynamics, EBM energy, SOC, MoE decoder, entity extraction, quality reflection.

What changes:

Subsystem	Default	Training	Why
Quantization	✅ 8-bit	❌ OFF	No compression during training — raw gradients needed
Graph	✅ ON	❌ OFF	Not relevant for model training
Semantic Memory	✅ ON	❌ OFF	Not relevant for model training
Batch Size	32	128	Larger batches for stable gradient estimates
Sequence Length	32	64	Longer contexts for contrastive learning
Learning Rate	1e-4	3e-4	Faster convergence with larger batches

Subsystem	Default	Training	Why
Noise Levels	5	7 [0.01..0.5]	More augmentation for robustness testing
Matryoshka Dims	[64,128,256,640]	[32,64,128,256,640]	Adds 32-dim for edge deployments
Epochs	N/A	50 with eval every 50 steps	Training loop active
Distillation	OFF	ON	Compress teacher→student knowledge
Data Lake	OFF	ON (10M entries)	Manages training datasets with compression
Entity N-grams	[2,3,4]	[2,3,4,5]	Wider n-gram capture for metadata
Entity Confidence	0.5	0.3	More permissive extraction during research
Max Splats	100K	500K	Room for evaluation datasets

Use when: training new embedding models, fine-tuning with noise robustness, Matryoshka representation learning, distilling large models into smaller ones, benchmarking embedding quality with `evaluate_embeddings` .

`M2MConfig::distributed(device)` — **Multi-Node Cluster**

Starts from default, then scales horizontally. Adds distributed indexing (MapReduce), auto-scaling (3–50 nodes), and aggressive compression for efficient inter-node data transfer. Keeps semantic search and quality monitoring.

What you get from default: HRM2 search, 3-tier memory, Langevin dynamics, EBM energy, SOC, MoE, entity extraction, quantization, quality reflection.

What changes:

Subsystem	Default	Distributed	Why
Auto-Scaling	OFF	3–50 nodes (<code>cooldown=20s</code>)	Dynamically add/remove nodes by load
MapReduce	8 chunks	32 chunks	Parallel indexing across nodes
Quantization	8-bit	4-bit	Smaller payloads between nodes
Search Fraction	0.1	0.2	Re-rank more candidates from shard merges
Quality Target	95%	97%	Higher bar for distributed recall
Semantic Decay	24h	2h	Faster staleness detection in distributed setting
Max Splats	100K	10M	Sharded across nodes
Init Splats	10K	500K	Pre-allocate for large datasets
Graph	 ON	 ON (inherited)	Cross-node graph traversal

What's NOT included (available via `advanced()`): HNSW secondary index, deep graph traversal, 1M+ per-node capacity, auto-scaling with aggressive compression.

Use when: deploying across 3+ servers, horizontal scaling for production, sharding large indices, load-balanced search clusters.

`M2MConfig::gpu(device)` — **GPU-Accelerated Full Pipeline**

Starts from advanced (everything ON), then adds GPU acceleration. This is the most complete preset — all features from advanced plus CUDA batch operations, GPU auto-tuning, and aggressive HNSW for GPU memory hierarchies.

What you get from advanced: TurboQuant 4-bit, GraphSplat deep, Semantic Memory RRF, HNSW, MapReduce, auto-scaling, quality 99%, EBM, Langevin, 3-tier memory, MoE.

What GPU adds on top:

Subsystem	Advanced	GPU	Why
Device	configurable	CUDA mandatory	GPU is the point
GPU Search	OFF	ON (<code>batch=4096</code> , auto-tune)	Batch distance on GPU, 10-100x throughput
HNSW M	32	48	More neighbors = better recall at GPU speed
HNSW ef_construction	400	800	GPU can afford higher build cost

Subsystem	Advanced	GPU	Why
HNSW ef_search	100	200	GPU searches faster, can explore more
Quant Projections	80	160	More projections for better compression at scale
Search Fraction	0.2	0.05	GPU re-ranks faster, can afford fewer candidates
Graph Neighbors	20	32	Larger graph = richer context
KNN k	128	256	More results per query
HRM2 Coarse/Fine	32/128	64/256	Finer granularity for 5M splats
HRM2 Probe	8	16	More clusters probed
Max Splats	1M	5M	GPU memory handles more
Init Splats	50K	100K	Pre-allocate for GPU

Use when: RTX 3090 or better available, real-time serving, batch search at scale, production with GPU. This preset subsumes advanced — use it instead of advanced when you have CUDA.

All Configurable Fields

```
[System]
device = "auto"           # auto | cpu | vulkan | cuda
dtype = "Float32"         # Float32 | Float64 | Int32 | Int64

[Latent Space]
latent_dim = 640
n_splats_init = 10000
max_splats = 100000
knn_k = 64

[Splat Parameters]
init_alpha = 1.0          # Initial opacity
init_kappa = 10.0         # Initial concentration
min_kappa = 1.0
max_kappa = 50.0

[Temperature & Energy]
splat_temperature = 0.1   # Exploration noise
weight_decay_start = 1.0
energy_splat_weight = 1.0 # Splat density energy
energy_geom_weight = 0.1  # Geometric regularization
energy_comp_weight = 0.0  # Composite (placeholder)
global_temperature = 1.0

[SOC - Self-Organized Criticality]
soc_threshold = 0.8
soc_buffer_capacity = 1000
soc_update_interval = 100
phi_convergence_threshold = 0.95

[HRM2 Search]
search_backend = "Hrm2"   # Hrm2 | Hnsw | Lsh | Quantized
hrm2_n_coarse = 16
hrm2_n_fine = 64
hrm2_n_probe = 4

[HNSW]
enable_hnsw = false
hnsw_m = 16
hnsw_ef_construction = 200
hnsw_ef_search = 50

[LSH]
enable_lsh = false
lsh_n_tables = 12
lsh_n_projections = 8
```



```
[Quantization]
enable_quantization = true
quant_algorithm = "TurboQuant"    # TurboQuant | PolarQuant | None
quant_bits = 8                    # 4 or 8 bits per dimension
quant_projections = 160
quant_seed = 42
quant_search_fraction = 0.1      # Fraction of candidates to re-rank

[GraphSplat]
enable_graph = true
graph_max_neighbors = 10
graph_traverse_depth = 3
graph_boost_weight = 0.05        # Score boost from graph neighbors

[Semantic Memory]
enable_semantic_memory = true
semantic_fusion = "Rrf"          # Rrf | Weighted | VectorOnly | Bm25Only
semantic_vector_weight = 0.7
semantic_bm25_weight = 0.3
semantic_bm25_k1 = 1.5
semantic_bm25_b = 0.75
semantic_decay_half-life = 86400.0 # seconds (24h)

[Langevin Dynamics]
langevin_steps = 200
langevin_dt = 0.001
langevin_gamma = 0.1
langevin_t = 1.0

[Context Windows]
context_local = 12
context_medium = 64
context_global = 512

[MoE Decoder]
vocab_size = 50257
hidden_dim = 1024
moe_experts = 4
moe_active = 2

[Training]
batch_size = 32
seq_length = 32
noise_levels = [0.01, 0.05, 0.1, 0.2, 0.5]

[Optimization]
learning_rate = 1e-4
weight_decay = 0.01
grad_clip = 1.0

[MapReduce Indexing]
enable_mapreduce = true
mapreduce_n_chunks = 8

[Auto-Scaling]
enable_auto_scaling = false
autoscale_min_nodes = 1
autoscale_max_nodes = 10
autoscale_cooldown_secs = 60.0

[Quality Reflection]
enable_quality_reflection = true
quality_recall_target = 0.95

[Training Mode]
enable_training = false
training_epochs = 10
training_eval_interval = 100
training_save_interval = 1000
```

```
training_noise_augmentation = true
training_matryoshka_dims = [64, 128, 256, 640]
training_distillation = false
```

```
[Data Lake]
enable_data_lake = false
data_lake_max_entries = 1_000_000
data_lake_compress = true
```

```
[Entity Extraction]
enable_entity_extraction = true
entity_ngram_sizes = [2, 3, 4]
entity_min_confidence = 0.5
```

```
[GPU Acceleration]
enable_gpu_search = false
gpu_batch_size = 1024
gpu_auto_tune = true
```

```
[API]
rest_port = 8080
grpc_port = 9090
```

CLI Commands

```
# GPU status and info
cargo run --features cuda -- gpu-info
# {"cuda_available":true,"device":"cuda","gpu_info":"NVIDIA GeForce RTX 3090 (sm_86), 25.8 GB total, 24.5 GB free"}

# Benchmark GPU vs CPU
cargo run --release --features cuda -- bench-gpu -n 100000 -d 640 -q 100 -k 10

# Quantization status
cargo run -- quant-status

# Vector search (SplatStore)
cargo run -- search --query "0.1,0.2,..." --top-k 10

# Energy check (SOC)
cargo run -- soc-check
```

License

[AGPL-3.0](#)